Backend Development Standart Stack For PT.LSKK

v1.0 — 20 Juli 2026. Dipecah dari dokumen Development Standart Stack (FE & BE) tanpa perubahan substansi aturan; kode yang sebelumnya berupa gambar ditranskrip menjadi teks. Bagian dari keluarga standar LSKK: Frontend Web, Backend, dan Mobile.

1. Mandatory Rules

- Lang : Typescript -> https://www.typescriptlang.org/
- Framework : Nest JS -> https://nestjs.com/
- Prettier Active Extensions : -> https://prettier.io/
- Database : MongoDB (https://www.mongodb.com/) & PostgreSQL -> https://www.postgresql.org/
- ODM (MongoDB) : Mongoose -> https://mongoosejs.com/
- ORM (PostgreSQL) : TypeORM -> https://typeorm.io/
- Caching Data : Nest Cache Manager -> https://docs.nestjs.com/techniques/caching
- MongoDB Transactional : Open Replicaset on database shell.
- TypeORM Transactional : Seems like Springboot style -> https://www.npmjs.com/package/typeorm-transactional
- DTO Validation : Class Transformer and Class Validation -> https://docs.nestjs.com/techniques/validation
- Must use data types, interfaces and enums. Minimize the use of any data types.
- Logger : Use Logger only from Nest Logger: must implements HTTP Logger in Middleware -> https://docs.nestjs.com/techniques/logger
- Websocket : using socket IO by Nest js -> https://docs.nestjs.com/websockets/gateways
- Realtime data One Way with HTTP : SSE (Server Sent Event) -> https://docs.nestjs.com/techniques/server-sent-events
- Rate Limiter : -> https://docs.nestjs.com/security/rate-limiting
- Cors : -> https://docs.nestjs.com/security/cors
- Authentication : Using JWT and Passport -> https://docs.nestjs.com/recipes/passport
- Authorization : Using Guard -> https://docs.nestjs.com/guards
- Testing : Nest Testing -> https://jestjs.io/

2. Catch Everythings With Filter

https://docs.nestjs.com/faq/request-lifecycle#filters

```
@Catch()
export class ExceptionFilter implements ExceptionFilter {
  constructor(private readonly httpAdapterHost: HttpAdapterHost) {}

  catch(exception: any, host: ArgumentsHost): void {
    const { httpAdapter } = this.httpAdapterHost;
```

```
    const context: HttpArgumentsHost = host.switchToHttp();
    const request: Request = context.getRequest();
    const response: Response = context.getResponse();

    const httpStatus: number =
      exception instanceof HttpException
        ? exception.getStatus()
        : HttpStatus.INTERNAL_SERVER_ERROR;

    const responseBody: {
      code: number;
      status: boolean;
      message: string;
      path: string;
    } = {
      code: httpStatus,
      status: false,
      message: exception.response?.message || exception.message,
      path: httpAdapter.getRequestUrl(request),
    };

    httpAdapter.reply(response, responseBody, httpStatus);
  }
}
```

3. Response Wrapper Interface

```
export interface IResponseEntity<T> {
  code: number;
  status: boolean;
  message: string;
  data?: T;
  meta?: ImetaPagination;
}
```

```
export interface ImetaPagination {
  totalPages: number;
  totalData: number;
  totalDataPerPage: number;
  page: number;
  limit: number;
}

export interface IResponsePageWrapper<T> {
  data: T[];
  meta: ImetaPagination;
}
```

4. Response Wrapper With Interceptors

https://docs.nestjs.com/faq/request-lifecycle#interceptors

```
@Injectable()
export class ResponseInterceptor implements NestInterceptor {
  constructor(private readonly messageService: MessageService) {}

  intercept(
    context: ExecutionContext,
    next: CallHandler<any>,
```

```
  ): Observable<any> {
    return next.handle().pipe(
      map((res: [] | { data: any; meta: ImetaPagination }) => {
        const response: IResponseEntity<any> = {
          code: context.switchToHttp().getResponse().statusCode,
          status: true,
          message:
            this.messageService.getMessage() || 'Successfully retrieve data',
        };
        if (res) {
          if (Array.isArray(res)) {
            response.data = res;
          } else if (res.meta) {
            response.data = res.data;
            response.meta = res.meta;
          } else {
            response.data = res;
          }
        }
        return response;
      }),
    );
  }
}
```

5. Next Development Upskilling?

- Next JS / Remix / Angular
- Make Google Account Developer LSKK For SSO integrating.
- Go lang / Rust lang For Microservices (Binary Output Executable, Consider use this lang for fast performance worker microservices)
- Docker Compose (Containerization your application to isolate versioning dependencies)
- CI / CD With Github Workflows (Consider use Github Workflows for automation Deployment application into server).
- Penetration Testing.
- Payment Gateway (Currently option is Midtrans)